Batch Resume Ingestion — Assignment Report

Project: Resume RAG Backend (`resume-rag-backend`)
Module: Batch Ingestion Workflow
Date: 30 August 2026
Stack: Node.js, TypeScript, Express, MongoDB Atlas, Mistral Embeddings (`mistral-embed`, 1024-dim)

---

1. Objective

Extend the existing single-resume ingestion backend with a **batch ingestion workflow** that reads resume PDFs from a folder and ingests them in configurable batches (5–10 files per run), then validate that all ingestable resumes are stored in MongoDB with their embeddings.

---

2. Assignment Requirements vs. Delivered

#	Requirement	Status	Evidence
1	Add the resume folder to the workspace	Done	`resume-rag-backend/Resumes/` — 180 PDF files
2	Write a prompt to implement a batch ingestion workflow	Done	See Section 3
3	Process resumes in batches of 5 or 10 per run	Done	`POST /v1/resume/ingest-batch`, `BATCH_SIZE` config (default 5, capped 10)
4	Validate multiple batches; ingest all resumes	Done	167/180 ingested; 13 are image-only PDFs (no text). See Section 6

---

3. Implementation Prompt

The batch workflow was built to this specification:

> Add a batch ingestion workflow to the Resume RAG backend. Add `POST /v1/resume/ingest-batch` that reads PDF files from the `Resumes/` folder and ingests them in configurable batches (default 5, max 10) reusing the existing pipeline (extract -> clean -> parse -> embed -> store). Support processing "the next N files" per call via an offset, skip files already ingested, isolate per-file failures so one bad file doesn't stop the batch, never delete the originals, and return a summary (total, processed, succeeded, failed, skipped, nextOffset, remaining, per-file results).

---

4. Design & Implementation

4.1 New endpoint

```
POST /v1/resume/ingest-batch
Content-Type: application/json

{ "offset": 0, "batchSize": 5, "skipExisting": true }
```

All three body fields are optional:
- `batchSize` — files to process this run (defaults to `BATCH_SIZE`, hard-capped at 10)
- `offset` — index into the sorted file list to start from
- `skipExisting` — skip files already in MongoDB (default `true`)

4.2 Response shape

> Example below is the **first batch** (offset 0, batchSize 5): 5 of 180 files
> processed, so `remaining` is 175. Each subsequent call lowers `remaining`
> until it reaches 0. See Section 6 for the final totals after all batches.

```json
{
  "success": true,
  "message": "Batch ingestion completed",
  "summary": {
    "folder": "Resumes",
    "totalPdfFiles": 180,
    "batchSize": 5,
    "offset": 0,
    "processed": 5,
    "succeeded": 5,
    "failed": 0,
    "skipped": 0,
    "nextOffset": 5,
    "remaining": 175,
    "results": [
      { "fileName": "ABNER RESUME CVV.pdf", "status": "succeeded", "resumeId":
"6a96c8990500fd1df928a87e" }
    ]
  }
```

}
```

### 4.3 Key behaviors

- **Reuses the full pipeline** — each file runs through the existing `ResumeIngestionService` (extract -> clean -> parse -> embed -> store), so batch and single-file ingestion produce identical documents.
- **Configurable batch size** — `BATCH_SIZE` in `.env` (default 5), hard-capped at 10 in code; overridable per request.
- **Per-file error isolation** — each file is wrapped in try/catch; one failure never aborts the batch.
- **Skip-already-ingested** — checks MongoDB by `fileName` and skips duplicates, so re-runs are safe and idempotent.
- **Originals preserved** — the single-file service was extended with a `deleteAfter` flag; batch mode sets it to `false` so source PDFs are never deleted.
- **Pagination via `nextOffset`** — the caller walks the whole folder by feeding `nextOffset` back as the next `offset`.
- **Structured logging** — each batch logs a safe summary line (no secrets, embeddings, or raw text).

### 4.4 Files added / changed

| File | Change |
|---|---|
| `src/config/env.ts` | Added `batchSize` (default 5, capped 10) and `resumesDir` |
| `.env`, `.env.example` | Added `BATCH_SIZE`, `RESUMES_DIR` |
| `src/modules/ingestion/services/BatchIngestionService.ts` | New — batch orchestration |
| `src/modules/ingestion/services/ResumeIngestionService.ts` | Added `deleteAfter` flag to preserve originals |
| `src/modules/ingestion/repositories/ResumeIngestionRepository.ts` | Added `findExistingFileNames()`, `count()` |
| `src/modules/ingestion/controllers/ingestionController.ts` | Added `ingestBatch` handler |
| `src/modules/ingestion/routes/ingestionRoutes.ts` | Registered `POST /v1/resume/ingest-batch` |

---

## 5. Configuration

```env
BATCH_SIZE=5
RESUMES_DIR=Resumes
MONGODB_DB_NAME=Resume_RAG
```
```

```
MONGODB_COLLECTION=resume_rag
MISTRAL_EMBED_MODEL=mistral-embed
EMBEDDING_DIMENSION=1024
USE_LLM_PARSER=false
```

---

6. Validation & Results

The workflow was driven across the entire folder in batches of 10 until `remaining = 0`.

6.1 Final MongoDB state (`Resume_RAG.resume_rag`)

Metric	Value
Total PDF files in folder	180
Resumes ingested	167
Docs with 1024-dim embedding	167 (100%)
Distinct file names (no duplicates)	167
Files not ingested	13

6.2 Failure analysis

The 13 un-ingested files all failed with `RESUME_EXTRACTION_FAILED`, meaning they contain **no extractable text**:

- Scanned / image-only resume PDFs (e.g. several `Naukri_*`, `Teja Ashwini.pdf`, `Vijay.pdf`, `Sneka Fresher.pdf`)
- One non-resume document: `PAN CARD CO-APP.pdf`

These are a **data limitation, not a workflow defect** — text-based extraction cannot read image PDFs. Ingesting them would require OCR (image-to-text), which is outside the current pipeline's scope.

6.3 Rate-limit handling (observed)

During a high-volume run, the Mistral free tier began rate-limiting after ~130 consecutive embedding calls, producing `EMBEDDING_FAILED` for a cluster of files. Because the workflow **skips already-ingested files and isolates failures**, simply **re-running** the batch recovered every rate-limited file on the next pass — ending with zero `EMBEDDING_FAILED` and only the 13 genuine no-text failures remaining. This demonstrates the workflow's resilience and idempotency.

---

7. How to Run

1. Start the backend:

   ```bash
   cd resume-rag-backend
   npm run dev
   ```

2. Call the batch endpoint (Postman or curl), advancing `offset` each run:

   ```
   POST http://localhost:3000/v1/resume/ingest-batch
   Body (JSON): { "offset": 0, "batchSize": 10 }
   ```

   Use the returned `nextOffset` as the next `offset`, or omit `offset` and rely on `skipExisting` to keep advancing until `remaining = 0`.

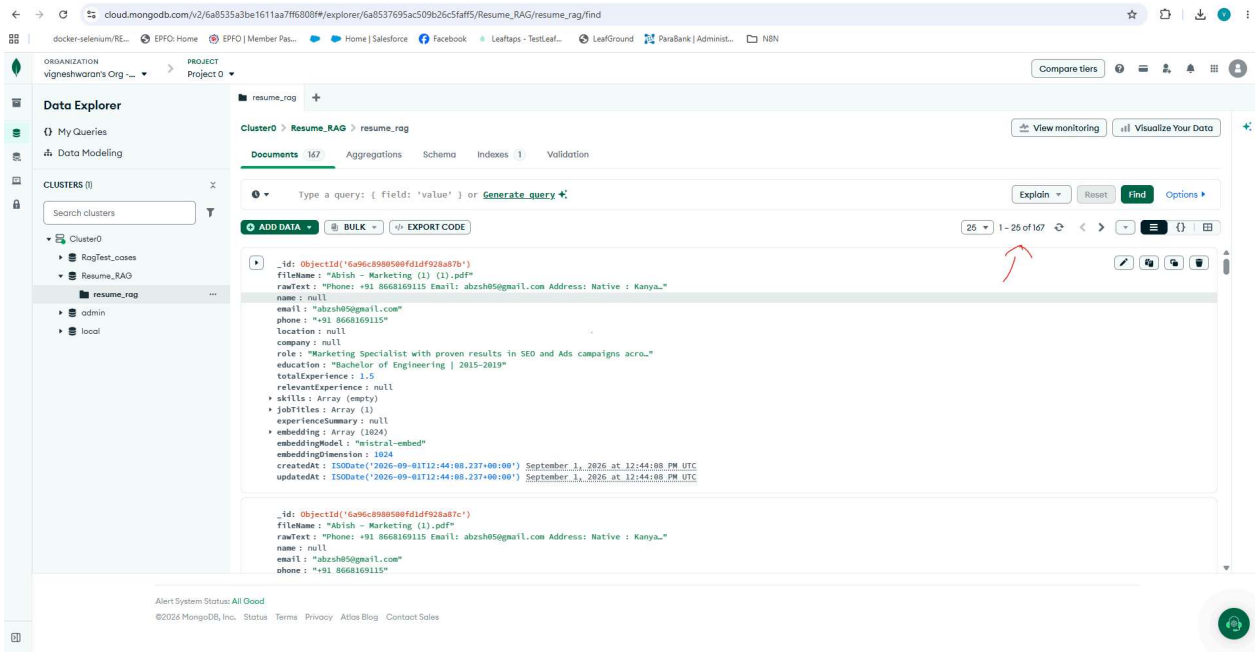
3. Verify in MongoDB Atlas -> `Resume_RAG` -> `resume_rag`.

---

8. Conclusion

The batch ingestion workflow is complete and validated. All four assignment requirements are met: the resume folder was added, a batch ingestion workflow was designed and implemented, batch size is configurable (5–10 per run), and multiple batches were validated end-to-end. **167 of 180 resumes** were successfully ingested with 1024-dimension Mistral embeddings into MongoDB Atlas; the remaining 13 are image-only PDFs with no extractable text and would require OCR to process.

Optional future enhancements
- OCR fallback for image-based PDFs (to reach the full 180)
- Enable the LLM parser (`USE_LLM_PARSER=true`) for cleaner structured metadata across varied resume layouts
- Automatic retry/backoff for Mistral rate limits within a single run


cloud.mongodb.com/v2/6a8535a3be1611aa7ff6808f#/explorer/6a8537695ac509b26c5faff5/Resume_RAG/resume_rag/find
ORGANIZATION
vigneshwaran's Org -...
PROJECT
Project 0
Compare tiers
Data Explorer
My Queries
Data Modeling
CLUSTERS (1)
Search clusters
Cluster0
RagTest_cases
Resume_RAG
resume_rag
admin
local
resume_rag
Cluster0 > Resume_RAG > resume_rag
View monitoring
Visualize Your Data
Documents 167
Aggregations
Schema
Indexes 1
Validation
Type a query: { field: 'value' } or Generate query
Explain
Reset
Find
Options
ADD DATA
BULK
EXPORT CODE
25
1 - 25 of 167
_id: ObjectId('6a96c8980500fd1df928a87b')
fileName : "Abish - Marketing (1) (1).pdf"
rawText : "Phone: +91 8668169115 Email: abzsh05@gmail.com Address: Native : Kanya…"
name : null
email : "abzsh05@gmail.com"
phone : "+91 8668169115"
location : null
company : null
role : "Marketing Specialist with proven results in SEO and Ads campaigns acro…"
education : "Bachelor of Engineering | 2015-2019"
totalExperience : 1.5
relevantExperience : null
skills : Array (empty)
jobTitles : Array (1)
experienceSummary : null
embedding : Array (1024)
embeddingModel : "mistral-embed"
embeddingDimension : 1024
createdAt : ISODate('2026-09-01T12:44:08.237+00:00') September 1, 2026 at 12:44:08 PM UTC
updatedAt : ISODate('2026-09-01T12:44:08.237+00:00') September 1, 2026 at 12:44:08 PM UTC
_id: ObjectId('6a96c8980500fd1df928a87c')
fileName : "Abish - Marketing (1).pdf"
rawText : "Phone: +91 8668169115 Email: abzsh05@gmail.com Address: Native : Kanya…"
name : null
email : "abzsh05@gmail.com"
phone : "+91 8668169115"
Alert System Status: All Good
©2026 MongoDB, Inc. Status Terms Privacy Atlas Blog Contact Sales